这应该是CS50笔记的最后一弹了，其C语言部分的最后一个lecture也是有关于Data structure的，今天写这些就当作复习了。

链表 LIST

```
typedef struct node
{
    int number;
    struct node *next;
} node;
```

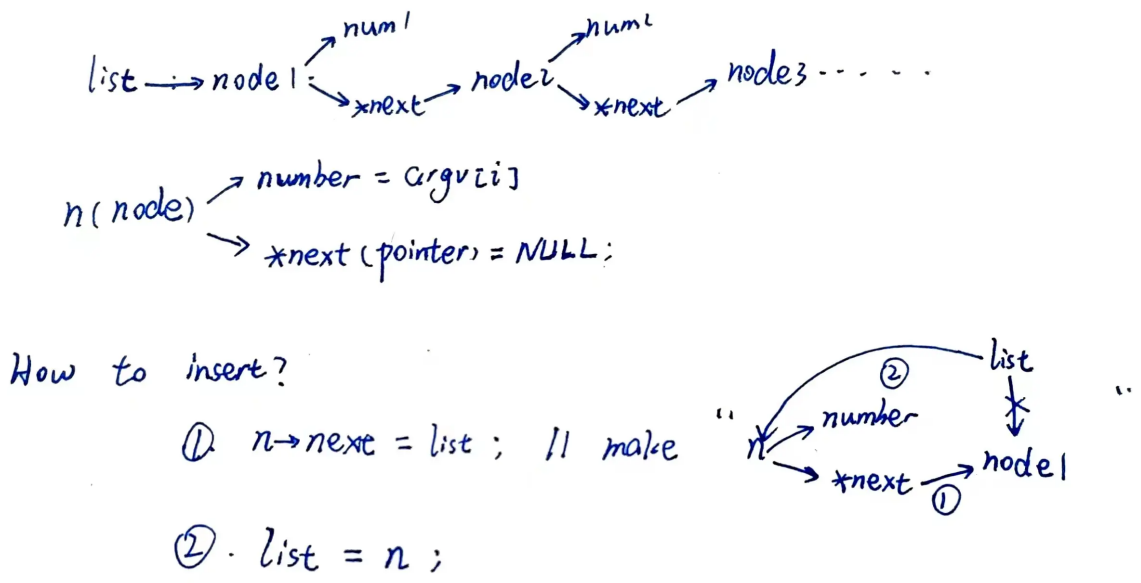
This is the basic stucture of LIST. Every node has a part which contains data, and a pointer which stores the address of the next node. So that these node can be linked together one by one, forming a chain.

Let's discuss th below code from Malan.

```
int main(int argc, char *argv[]) // 加入命令行参数
{
    node *list = NULL; // 定义了一个空链表，即元素数量为0的链表

    for (int i = 1; i < argc; i++) // argv的第一个值是 程序的名称
    {
        int number = atoi(argv[i]); // atoi 将ASCII码转换为整数，只要你输入的是
数字

        node *n = malloc(sizeof(node));
        if (n == NULL)
        {
            return 1;
        }
        n->number = number;
        n->next = NULL;

        n->next = list;
        list = n;
    }

    // node *ptr = list;
    // while (ptr != NULL)
    for (node *ptr = list; ptr != NULL; ptr = ptr->next) // 和上面的两行是等价
的
    {
        printf("%i\n", ptr->number);
    }

    // 开始释放
    node *ptr = list; // 回到开头
    while (ptr != NULL)
```

```
    {
        node *next = ptr->next;
        free(ptr);
        ptr = next;
    }
    return 0;
}
```

This small program can create a LIST with numbers based on the input of the commandline arguments, and print them from front to back. After performing these operations, the memory can be freed all.



This photo shows that `HOW TO INSERT A NODE INTO A LIST`.

Generally, if we want to read a LIST, we need a temporary pointer(like `ptr`)to traverse the entire chain, until it reaches the node we need.

```
// node *ptr = list;
// while (ptr != NULL)
    for (node *ptr = list; ptr != NULL; ptr = ptr->next) // 和上面的两行是等价的
    {
        printf("%i\n", ptr->number);
    }
```

`node *ptr = list` is to create a temporary pointer`ptr`. `ptr != NULL` serves as a limitation, preventing access to prohibited memory or handling cases where the memory allocated for ptr was not available during

its creation. `ptr = ptr->next` enables us to obtain the next node in the sequence after the current one.

A very important thing is the freeing of the linked list. This process relies on the temp pointer, but it also involves another pointer, `next`. Nodes are deleted from the front to the back one by one.

```
node *ptr = list; // 回到开头
while (ptr != NULL)
{
    node *next = ptr->next; // 先get下一个node
    free(ptr);
    ptr = next;
}
```

Pros:

1. Dynamic Memory Allocation: We don't need a complete memory space; instead, the structure allows us to adjust the amount of data by creating or deleting nodes. Knowing the total number of data elements in advance is unnecessary, and there is no pre-allocation of space.
2. Efficient Insertion and Deletion: Due to the pointer relationships between nodes, inserting and deleting nodes in a linked list is more efficient than in arrays. The adjustment mostly relies on changing the pointer.
3. Minor Impact: When you change a node, the only nodes on either side of it will be impacted.

Cons:

1. Memory Overhead/Extra Storage for Pointers: Each node in a linked list contains both data and a pointer to the next node. This additional overhead consumes more memory compared to arrays, where only the data needs to be stored.
2. Sequential Access: Unlike arrays, linked lists don't provide constant-time access to individual elements. To access an element, you must start from the head (or another known node) and traverse the whole list sequentially. This makes random access time, especially for a large list.

`Doubly-Linked Lists` has a more pointer to contain the address of the previous node.

Trees 🌲

The stucture of a tree contains many nodes, and every node has three parts`*left *right data`.

Similarities:

1. Nodes.
2. Pointers.

Differences:

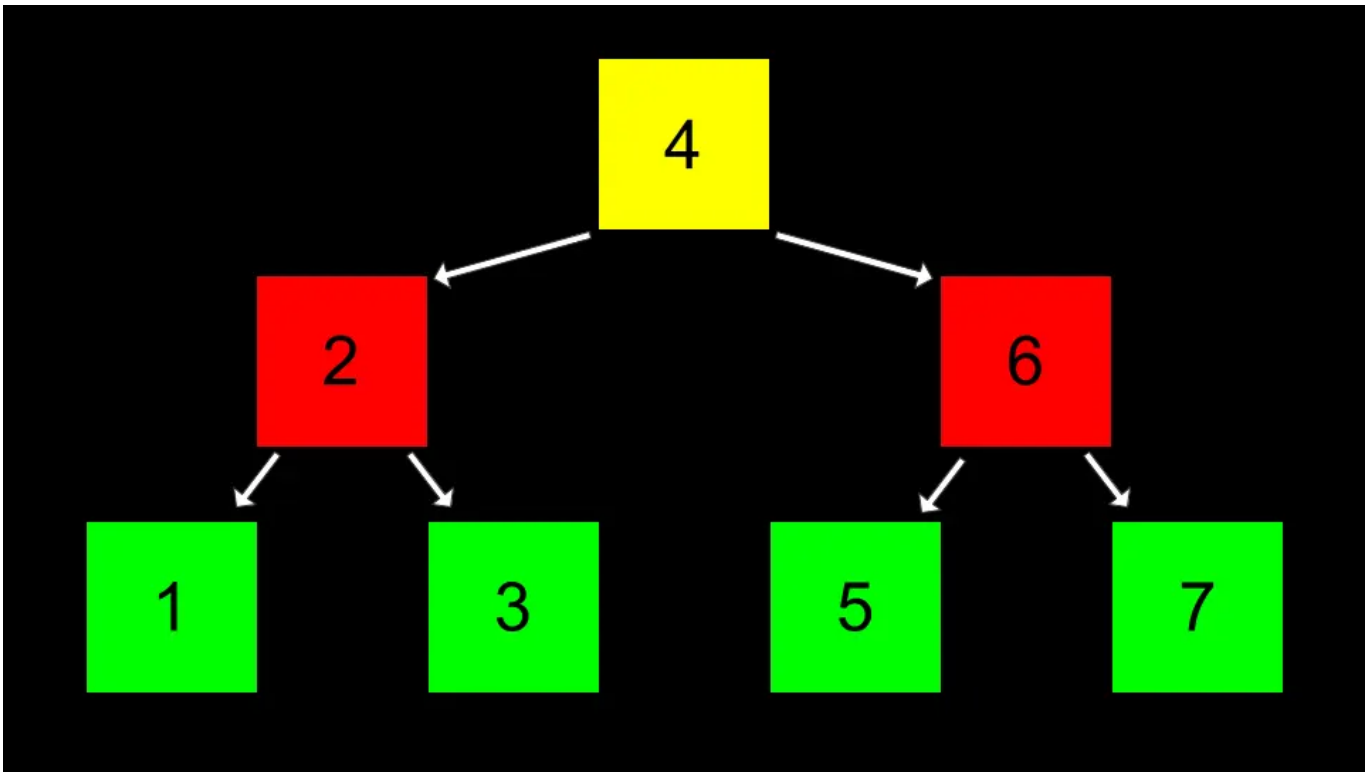
1. Hierarchical Structure: A tree is **a hierarchical data structure**, comprising a root node, internal nodes, and leaf nodes. The connections between nodes form a tree-like structure. In contrast, a

linked list is **a linear structure**, where nodes have only predecessors and successors, creating a linear sequence.

2. Node Relationships: In a linked list, each node usually has **a single successor**. In a tree, nodes can have **multiple child nodes**, establishing parent-child relationships. A node's parent is the node directly above it, and its children are the nodes directly below.
3. Traversal Patterns: Linked lists are typically traversed **linearly**, from the head to the tail or vice versa. Trees can be traversed **in various ways**, such as pre-order, in-order, and post-order traversals.

Binary search trees are another data structure that can be used to store data more efficiently such that it can be searched and retrieved.

If the BST has a balance, the time complexity is O(log n) for all cases.




```
bool search(node *tree, int number)
{
    if (tree == NULL)
    {
        return false;
    }
    else if (number < tree->number)
    {
        return search(tree->left, number);
    }
    else if (number > tree->number)
    {
        return search(tree->right, number);
    }
    else if (number == tree->number)
    {
        return true;
```

```
    }
}
```

Hashing and Hash Tables 哈希化 和 哈希表

Hashing is the idea of taking a value and being able to output a value that becoms a shortcut to it later. **A hash table** is a fantastic combination of both arrays and linked lists. When implemented in code, a hash table is an array of pointers to nodes. **A hash function** is an algorithm that reduces a larger value to something small and predictable. Generally, this function takes in an item you wish to add to your hash table, and returns an integer representing the array index in which the item should be placed.

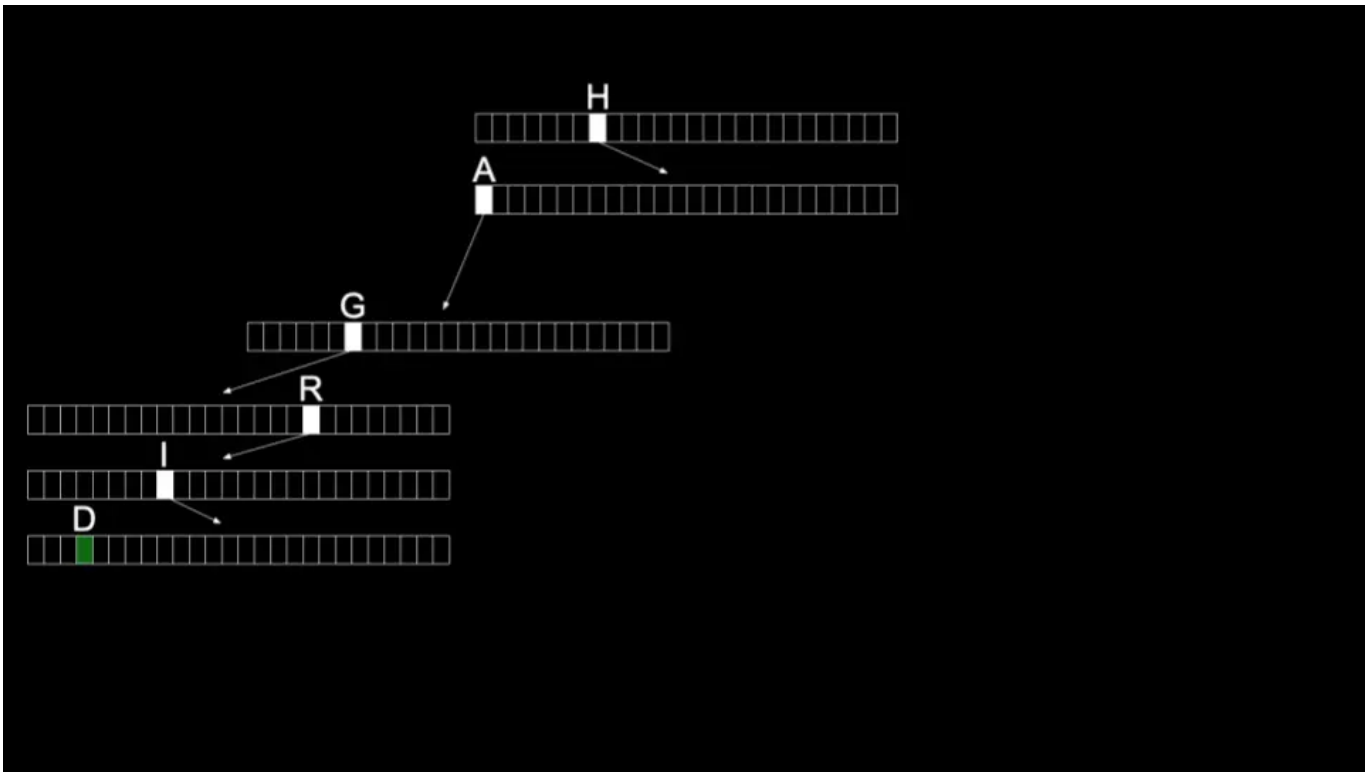
For example, hasing *apple* may hash as a value of 1, and *berry* may be hashed as 2. Therefore, finding *apple* is as easy as aksing the hashed algorithm where *apple* is stored. However, if we have a lot of words to store, relying solely on the first character to hash them is inefficient and not ideal. In such cases, it becomes necessary to create a larger hash table.

You, as the programmer, have to make a decision about the advantages of ***using more memory to have a large hash table and potentially reducing search time*** *or* ***using less memory and potentially increasing search time.***

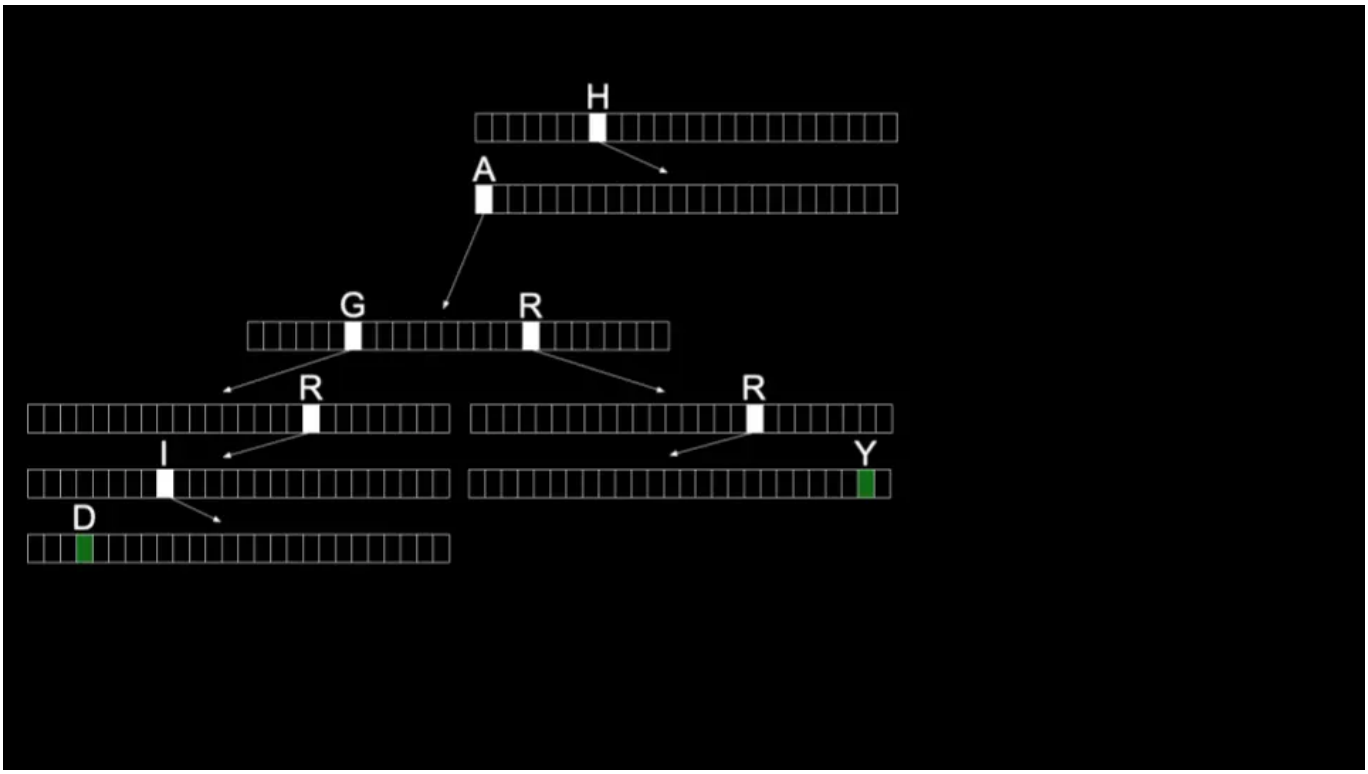
Tries 🌲

Tries are always searchable in constant time.

One downside to Tries is that they tend to take up a large amount of memory. Notice that we need 26 X 5 = 120 nodes just to store *Hagrid!*



Harry would then be stored as follows:



Goodbye, CS50! Have a nice day!

Mostly_Harmless